SOLAR POWERED BLIND CURVE WARNING SYSTEM

Arduino UNO source code for GitHub project upload

1. Project overview

This example uses two HC-SR04 ultrasonic sensors to detect vehicles approaching a blind curve from opposite directions. When Sensor A detects a vehicle, the warning LED and buzzer on Side B are activated. Sensor B similarly activates the warning on Side A. The system can be powered from a solar panel through a suitable charge controller and regulated battery supply.

2. Components required

Component	Quantity
Arduino UNO	1
HC-SR04 ultrasonic sensor	2
Red LED with suitable series resistor	2
Active buzzer module (5 V)	2
Jumper wires and breadboard / PCB	As required
Solar panel, charge controller, battery and regulated 5 V supply	1 set

3. Pin connections

Arduino pin	Connection
D2 / D3	Sensor A TRIG / ECHO
D4 / D5	Sensor B TRIG / ECHO
D6 / D7	Side A warning LED / buzzer
D8 / D9	Side B warning LED / buzzer
5V and GND	Sensor and alert-module supply / common ground

Important: connect each LED through a current-limiting resistor. Use a driver transistor or suitable driver module if a buzzer draws more current than an Arduino pin can safely supply. Do not connect a solar panel or battery directly to the Arduino 5V pin; use a proper charge controller and regulated supply.

4. Arduino code

Copy the complete code below into the Arduino IDE, select Arduino UNO, verify, and upload.

```
// Solar Powered Blind Curve Warning System
// Board: Arduino UNO
// Sensors: 2 x HC-SR04 ultrasonic sensors
// Alerts: 2 x red LEDs and 2 x buzzers
//
// Sensor A detects a vehicle approaching from Side A and warns Side B.
// Sensor B detects a vehicle approaching from Side B and warns Side A.

const byte TRIG_A = 2;
const byte ECHO_A = 3;
const byte TRIG_B = 4;
const byte ECHO_B = 5;

const byte LED_A = 6;       // Warning for vehicles on Side A
const byte BUZZER_A = 7;
const byte LED_B = 8;       // Warning for vehicles on Side B
const byte BUZZER_B = 9;

const float DETECT_DISTANCE_CM = 150.0; // Adjust after testing
const unsigned long SENSOR_TIMEOUT_US = 25000UL;

float readDistanceCM(byte trigPin, byte echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  unsigned long duration = pulseIn(echoPin, HIGH, SENSOR_TIMEOUT_US);

  // No echo received: return a value meaning "no vehicle detected".
  if (duration == 0) {
    return 999.0;
  }

  return (duration * 0.0343) / 2.0;
}

void setup() {
  pinMode(TRIG_A, OUTPUT);
  pinMode(ECHO_A, INPUT);
  pinMode(TRIG_B, OUTPUT);
  pinMode(ECHO_B, INPUT);

  pinMode(LED_A, OUTPUT);
  pinMode(BUZZER_A, OUTPUT);
  pinMode(LED_B, OUTPUT);
  pinMode(BUZZER_B, OUTPUT);

  digitalWrite(TRIG_A, LOW);
  digitalWrite(TRIG_B, LOW);

  Serial.begin(9600);
}

void loop() {
  float distanceA = readDistanceCM(TRIG_A, ECHO_A);
  delay(50); // Reduces interference between ultrasonic sensors
  float distanceB = readDistanceCM(TRIG_B, ECHO_B);

  // A vehicle on Side A warns traffic approaching from Side B.
  bool warnSideB = (distanceA > 2.0 &&
                    distanceA <= DETECT_DISTANCE_CM);

  // A vehicle on Side B warns traffic approaching from Side A.
  bool warnSideA = (distanceB > 2.0 &&
                    distanceB <= DETECT_DISTANCE_CM);

  digitalWrite(LED_A, warnSideA ? HIGH : LOW);
  digitalWrite(BUZZER_A, warnSideA ? HIGH : LOW);

  digitalWrite(LED_B, warnSideB ? HIGH : LOW);
  digitalWrite(BUZZER_B, warnSideB ? HIGH : LOW);

  Serial.print("Side A distance: ");
  Serial.print(distanceA);
  Serial.print(" cm | Side B distance: ");
  Serial.print(distanceB);
  Serial.println(" cm");

  delay(100);
}
```

5. Testing and adjustment

• Upload the sketch and open Serial Monitor at 9600 baud.

• Move an object in front of Sensor A. Side B's LED and buzzer should turn on.

• Move an object in front of Sensor B. Side A's LED and buzzer should turn on.

• Change DETECT_DISTANCE_CM to suit the road layout and sensor mounting distance.

• Test outdoors in the actual mounting position. Ultrasonic readings can be affected by rain, wind, object angle and vehicle shape.

6. GitHub upload steps

• Create a new repository on GitHub, for example: solar-blind-curve-warning-system.

• Save the Arduino sketch as Solar_Blind_Curve_Warning_System.ino.

• Open the repository and choose Add file → Upload files.

• Upload the .ino file and this PDF documentation, then select Commit changes.

• Add photos, circuit diagram and test results to the repository if available.

Note

This is a basic prototype program for academic demonstration, not a certified road-safety device. A real installation needs weatherproof enclosures, reliable vehicle sensing, fail-safe warning logic, field testing and approval from the relevant road authority.